Generics in Java

Last Updated : 22 Nov, 2025

Generics means parameterized types. They allows us to write code that works with different data types using a single class, interface or method. Instead of creating separate versions for each type, we use type parameters (like <T>) to make the code reusable and type-safe.

Why Use Generics?

- Before Generics, Java collections like ArrayList or HashMap could store any type of object, everything was treated as an Object. It had some problems.
- If you added a String to a List, Java didn't remember its type. You had to manually cast it when retrieving. If the type was wrong, it caused a runtime error.
- With Generics, you can specify the type the collection will hold like ArrayList<String>. Now, Java knows what to expect and it checks at compile time, not at runtime.

Types of Java Generics

1. Generic Class

A generic class is a class that can operate on objects of different types using a type parameter. Like C++, we use <> to specify parameter types in generic class creation. To create objects of a generic class, we use the following syntax:

// To create an instance of generic class
BaseType <Type> obj = new BaseType <Type>()

Note: In Parameter type, we can not use primitives like "int", "char" or "double". Use wrapper classes like Integer, Character, etc.

```
// We use < > to specify Parameter type
class Test<T> {

    T obj;
    Test(T obj) {
        this.obj = obj;
    }
    public T getObject() { return this.obj; }
}

class Geeks {
    public static void main(String[] args)
    {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(15);
        System.out.println(iObj.getObject());

        // instance of String type
        Test<String> sObj
```

```
            = new Test<String>("GeeksForGeeks");
        System.out.println(sObj.getObject());
    }
}
```

Output

```
15
GeeksForGeeks
```

We can also pass multiple Type parameters in Generic classes.

```
class Test<T, U>
{
    T obj1;  // An object of type T
    U obj2;  // An object of type U

    Test(T obj1, U obj2)
    {
        this.obj1 = obj1;
        this.obj2 = obj2;
    }

    public void print()
    {
        System.out.println(obj1);
        System.out.println(obj2);
    }
}

class Geeks
{
    public static void main (String[] args)
    {
        Test <String, Integer> obj =
            new Test<String, Integer>("GfG", 15);

        obj.print();
    }
}
```

Output

```
GfG
15
```

2. Generic Method

We can also write generic methods that can be called with different types of arguments based on the type of arguments passed to the generic method. The compiler handles each method.

```
class Geeks {

    // A Generic method example
    static <T> void genericDisplay(T element)
    {
        System.out.println(element.getClass().getName()
```

```
                                + " = " + element);
    }

    public static void main(String[] args)
    {
        // Calling generic method with Integer argument
        genericDisplay(11);

        // Calling generic method with String argument
        genericDisplay("GeeksForGeeks");

        // Calling generic method with double argument
        genericDisplay(1.0);
    }
}
```

Output

```
java.lang.Integer = 11
java.lang.String = GeeksForGeeks
java.lang.Double = 1.0
```

Limitations of Generics

1. Generics Work Only with Reference Types

When we declare an instance of a generic type, the type argument passed to the type parameter must be a reference type. We cannot use primitive data types like **int**, **char.**

Test<int> obj = new Test<int>(20);

The above line results in a compile-time error that can be resolved using type wrappers to encapsulate a primitive type.

But primitive type arrays can be passed to the type parameter because arrays are reference types.

ArrayList<int[]> a = new ArrayList<>();

2. Generic Types Differ Based on their Type Arguments

During compilation, generic type information is erased which is also known as type erasure.

```
class Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) { this.obj = obj; } // constructor
    public T getObject() { return this.obj; }
}

class Geeks {
    public static void main(String[] args)
    {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(15);
        System.out.println(iObj.getObject());

        // instance of String type
        Test<String> sObj
            = new Test<String>("GeeksForGeeks");
        System.out.println(sObj.getObject());
        iObj = sObj; // This results an error
    }
}
```

Output:

```
error:
 incompatible types:
 Test cannot be converted to Test
```

Explanation: Even though iObj and sObj are of type Test, they are the references to different types because their type parameters differ. Generics add type safety through this and prevent errors.

Type Parameter Naming Conventions

The type parameters naming conventions are important to learn generics thoroughly. The common type parameters are as follows:

- T: Type
- E: Element
- K: Key
- N: Number
- V: Value

Benefits of Generics

Programs that use Generics has got many benefits over non-generic code.

1. Code Reuse: We can write a method/class/interface once and use it for any type we want.

2. Type Safety: Generics make errors to appear compile time than at run time (It's always better to know problems in your code at compile time rather than making your code fail at run time).

Suppose you want to create an ArrayList that store name of students and if by mistake the programmer adds an integer object instead of a string, the compiler allows it. But, when we retrieve this data from ArrayList, it causes problems at runtime.

Example: Without Generics

```java
import java.util.*;

class Geeks
{
    public static void main(String[] args)
    {
        // Creatinga an ArrayList without any type specified
        ArrayList al = new ArrayList();

        al.add("Sweta");
        al.add("Gudly");
        al.add(10); // Compiler allows this

        String s1 = (String)al.get(0);
        String s2 = (String)al.get(1);

        // Causes Runtime Exception
        String s3 = (String)al.get(2);
    }
}
```

Output :

```
Exception in thread "main" java.lang.ClassCastException:
   java.lang.Integer cannot be cast to java.lang.String
    at Test.main(Test.java:19)
```

Here, we get runtime error.

How do Generics Solve this Problem?

When defining ArrayList, we can specify that this list can take only String objects.

Example: With Generics

```
import java.util.*;

class Geeks
{
    public static void main(String[] args)
    {
        // Creating a an ArrayList with String specified
        ArrayList <String> al = new ArrayList<String> ();

        al.add("Sweta");
        al.add("Gudly");

        // Now Compiler doesn't allow this
        al.add(10);

        String s1 = (String)al.get(0);
        String s2 = (String)al.get(1);
        String s3 = (String)al.get(2);
    }
}
```

Output:

```
15: error: no suitable method found for add(int)
        al.add(10);
          ^
```

3. Individual Type Casting is not needed: If we do not use generics, then, in the above example, every time we retrieve data from ArrayList, we have to typecast it. Typecasting at every retrieval operation is a big headache. If we already know that our list only holds string data, we need not typecast it every time.

```
import java.util.*;

class Geeks {
    public static void main(String[] args)
    {
        // Creating a an ArrayList with String specified
        ArrayList<String> al = new ArrayList<String>();

        al.add("Sweta");
        al.add("Gudly");

        // Typecasting is not needed
        String s1 = al.get(0);
        String s2 = al.get(1);
    }
}
```

4. Generics Promotes Code Reusability: With the help of generics in Java, we can write code that will work with different types of data. For example, Let's say we want to Sort the array elements of various data types like int, char, String etc. Basically we will be needing different functions for different data types. For simplicity, we will be using Bubble sort.

Example: Generic Sorting

```
public class Geeks {

    public static void main(String[] args)
    {
        Integer[] a = { 100, 22, 58, 41, 6, 50 };

        Character[] c = { 'v', 'g', 'a', 'c', 'x', 'd', 't' };
```

```java
        String[] s = { "Amiya", "Kuna", "Gudly", "Sweta","Mama", "Rani", "Kandhei" };

        System.out.print("Sorted Integer array:  ");
        sort_generics(a);

        System.out.print("Sorted Character array:  ");
        sort_generics(c);

        System.out.print("Sorted String array:  ");
        sort_generics(s);
    }

    public static <T extends Comparable<T> > void sort_generics(T[] a)
    {
          //As we are comparing the Non-primitive data types we need to use Comparable class

        //Bubble Sort logic
        for (int i = 0; i < a.length - 1; i++) {

            for (int j = 0; j < a.length - i - 1; j++) {

                if (a[j].compareTo(a[j + 1]) > 0) {

                    swap(j, j + 1, a);
                }
            }
        }
        // Printing the elements after sorted
        for (T i : a)
        {
            System.out.print(i + ", ");
        }
        System.out.println();

    }

    public static <T> void swap(int i, int j, T[] a)
    {
        T t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
}
```

Output

```
Sorted Integer array:  6, 22, 41, 50, 58, 100,
Sorted Character array:  a, c, d, g, t, v, x,
Sorted String array:  Amiya, Gudly, Kandhei, Kuna, Mama, Rani, Sweta,
```

Here, we have created a generics method. This same method can be used to perform operations on integer data, string data and so on.

> ***Note**: If you are new to Java, start practicing Generics with basic examples like generic Box, generic Pair and generic methods.*

Generics in Java

Visit Course

Suggested Quiz

4 Questions

Quiz Completed Successfully

Your Score :3/4 Accuracy :75%

Comment

K **kartik** 419

Article Tags: Java Java-Generics

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training

Explore
POTD
Practice Problems
Connect
Blogs
Upskill Courses

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Offline Centers
Noida
Bengaluru
Pune
Hyderabad
Kolkata

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved